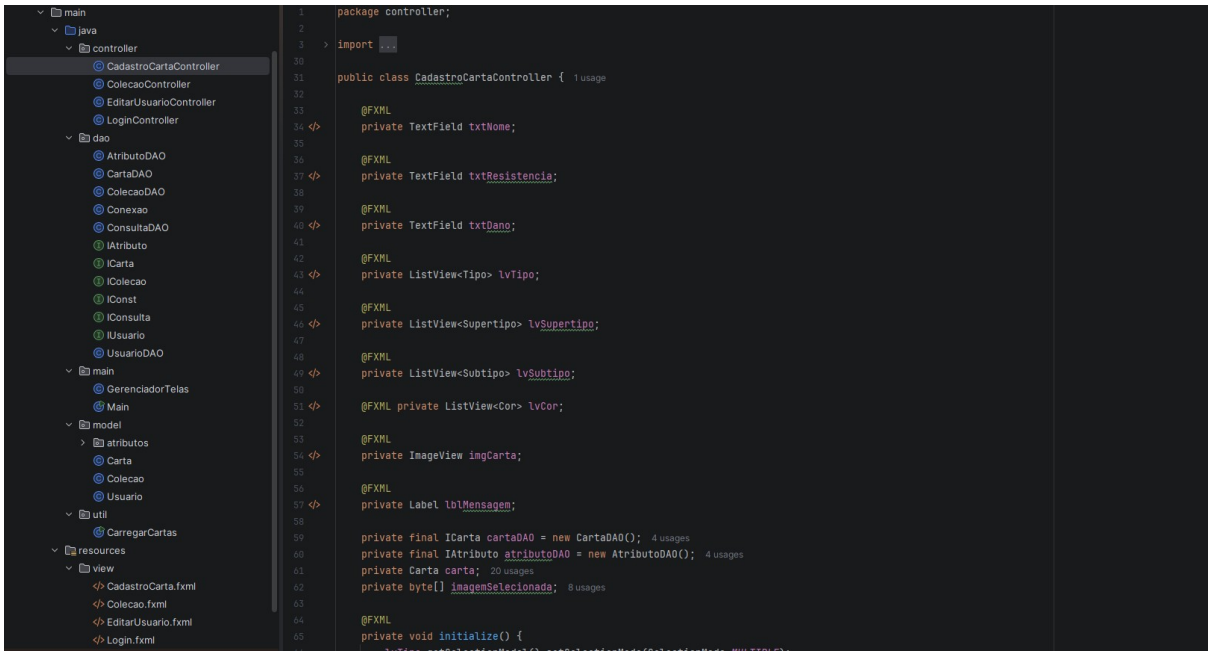
2º Trabalho de POO II: Aplicação de padrões de projeto

1. Diagnóstico do projeto original

Este trabalho usa como base o Magic Card Tracker, software desenvolvido como primeiro trabalho da disciplina.



No projeto inicial, certas classes estavam sobrecarregadas, havia pouca modularização e o projeto em geral estava lotado de redundâncias.

2. Padrões Aplicados

Padrão	Service

Problema	Cada Controller tratava os erros SQL diretamente, acumulando tarefas e repetindo código entre as classes.
	
	exemplo:
	`public void btnAcessarOnAction() {`
	`    ...`
	`    try {`
	`        usuario = usuarioDAO.acessarUsuario(nome);`
	`    } catch (SQLException e) {`
	`        erro("Erro no sistema: tente novamente mais tarde!");`
	`        return;`
	`    }`
	`    if (usuario == null) {`
	`        erro("Login Invalido: usuario não cadastrado!");`
	`    }`
	`    ...`
	`}`

Solução	Criar classes intermediárias (Services) entre DAO e Controller, que fornecem os dados ao controller e tratam internamente os erros SQL.
	
	exemplo_controller:
	`try {`
	`    usuario = usuarioService.autenticar(txfNome.getText(),`
	`                                        psfSenha.getText());`

	`} catch (ServicoException e) {`
	`  erro("Login Inválido: " + e.getMessage());`
	`  return;`
	`}`
	
	exemplo_service:
	`public Usuario autenticar(String nome, String senha)`
	`  throws ServicoException {`
	`  ...`
	`  } catch (SQLException e) {`
	`  throw new ServicoException(`
	`  "Erro no sistema: tente novamente mais tarde!", e);`
	`  }`
	`  if (usuario == null) {`
	`  throw new ServicoException("Usuário não cadastrado!");`
	`  }`
	`  ...`
	`}`

Justificativa e verificação	Os controllers deixaram de tratar SQLException e só exibem a mensagem da ServicoException, enquanto as regras ficaram centralizadas nos services. O uso pode ser verificado no LoginController, que chama usuarioService.autenticar(...).

Padrão	**Strategy**

Problema	O ConsultaDAO tinha um método para cada forma de buscar cartas (consultaTem e consultaFalta), com SQL duplicado guardado em uma String mutável.
	
	exemplo:
	`public List<Carta> consultaTem(Usuario usuario)`
	`  throws SQLException {`
	`  sql = " SELECT ... FROM temCarta t JOIN ... " + sql;`
	`  return executar();`
	`}`
	`// consultaFalta: mesma ideia, mas com EXCEPT`

Solução	Criar a interface EscopoColecao, implementada por CartasDoUsuario e CartasFaltantes. O ConsultaDAO (contexto) recebe a estratégia por parâmetro.
	
	exemplo:
	`public interface EscopoColecao {`
	`  String condicaoSql();`
	`}`
	
	`// ColecaoService`
	`consultaDAO.buscar(usuario, new CartasDoUsuario(), filtro);`
	`consultaDAO.buscar(usuario, new CartasFaltantes(), filtro);`

Justificativa e verificação	Um novo escopo de busca é só uma nova classe, sem alterar o ConsultaDAO. O uso pode ser verificado no ColecaoService.carregar, que chama buscar() com cada estratégia.

Padrão	**Builder**
Problema	O controller chamava um método de filtro por vez no ConsultaDAO,

	que acumulava o SQL em uma String, dependia da ordem das chamadas e concatenava os valores digitados.
	
	exemplo:
	`consulta.consultaNome(nome);`
	`consulta.consultaTipo(tipos);`
	`consulta.consultaDano(Integer.parseInt(dano));`

Solução	Criar a classe imutável FiltroCartas, construída por um Builder interno em que todos os filtros são opcionais.
	
	exemplo:
	`FiltroCartas filtro = FiltroCartas.builder()`
	`.nome(txtNome.getText())`
	`.dano(lerInteiro(txtDano))`
	`.build();`

Justificativa e verificação	O controller monta o filtro de uma vez, sem depender de ordem, e o ConsultaDAO passou a usar PreparedStatement. O uso pode ser verificado no ColecaoController.montarFiltro.

Padrão	**Command**

Problema	O controller chamava o DAO diretamente para adquirir e remover cartas, sem nenhum registro e sem como desfazer.
	
	exemplo:
	`colecaoDAO.adicionarCarta(usuarioLogado.getId(),`
	`carta.getId());`

Solução	Cada ação virou um objeto Comando, com executar e desfazer. A classe HistoricoComandos guarda os executados em uma pilha e é usada pelo ColecaoService.
	
	exemplo:
	`public class AdquirirCartaComando implements Comando {`
	`public void executar() throws SQLException {`
	`colecaoDAO.adicionarCarta(usuarioId, cartaId);`
	`}`
	
	`public void desfazer() throws SQLException {`
	`colecaoDAO.removerCarta(usuarioId, cartaId);`
	`}`
	`}`

Justificativa e verificação	Permitiu criar o botão Desfazer sem alterar as classes existentes, e novas ações são só novas classes. O uso pode ser verificado no ColecaoController.btnDesfazerOnAction.